Claude prompt caching: measure reuse before adding keepwarm

The shared mechanism

Caching reuses an eligible input prefix. It does not reuse a completed answer, remove output charges or guarantee a headline saving on the whole task. Stable instructions and reference context belong before frequently changing content. Prefix edits and expiry can cause new writes; output, tools and unrelated calls remain part of the bill. Use the current caching documentation (https://platform.claude.com/docs/en/build-with-claude/prompt-caching), with the selected model's eligibility and pricing, rather than a reel's fixed percentage.

Two applications of the same idea

The caching reel explains API prefix reuse. The Cache Tax reel adds a Claude Code monitor and keepwarm proposals. API settings, subscription accounting and provider behavior are different contexts: do not assume a single universal TTL or convert subscription usage into an exact dollar bill using API list prices.

The Cache Tax repository (https://github.com/karanb192/cache-tax) and claude-thermos (https://github.com/izeigerman/claude-thermos) are community tools. Follow their current READMEs after inspecting permissions; the older installation examples remain in their source guide. Keeping a context warm issues requests and may consume plan usage. Whether this is beneficial depends on actual reuse intervals and provider terms. No keepwarm process was installed or started here.

Reproducible experiment

1. Freeze a long, eligible prefix and record its digest, model, date and cache settings.
2. Make the initial request and retain the provider's reported input/cache-write/output fields.
3. Repeat without prefix edits and record reported cache reads.
4. Change a prefix token, repeat after expiry, and compare those cases.
5. Add all requests, output, tool calls and any warmup traffic to a ledger.
6. Compare the total against a no-cache baseline with identical successful tasks.

Check cost optimization guidance (https://platform.claude.com/docs/en/about-claude/models/optimizing-for-cost-and-intelligence) before choosing a smaller model or batch processing; those are separate changes that need their own quality comparison. Minimum cacheable lengths and price multipliers depend on the current model, so this guide deliberately does not freeze a single number.

Proposed project and prompt

Build a prefix/TTL simulator and cost ledger with fixture usage records, then an optional API adapter. Export a dated report of warm/cold cases and measured totals. The simulator should test changed-prefix invalidation, timeout handling and zero-hit cases; it must not label simulated savings as observed savings.

```
Create a cache ledger experiment using fixture records first. Keep model
pricing in a dated configurable table. Track ordinary input, cache writes,
cache reads, outputs and keepwarm traffic separately. Add an optional official
API adapter, tests for expiry and prefix changes, and a reproducible report.
Do not start a background keepwarm service or claim savings without a baseline.
```

The original sources and their PDFs are retained. This guide resolves their overlap and makes the remaining quota and provider assumptions explicit.

Original sources

- https://www.instagram.com/reel/DeUDrCLhi3H/
- https://www.instagram.com/reel/DdwqtoNIGus/